

IPO-1202: RAPPORT
A3P(AL) (2024-2025 G10)
Titre: All Aboard for Murder



I.A) Auteur: Youssef Shalaby

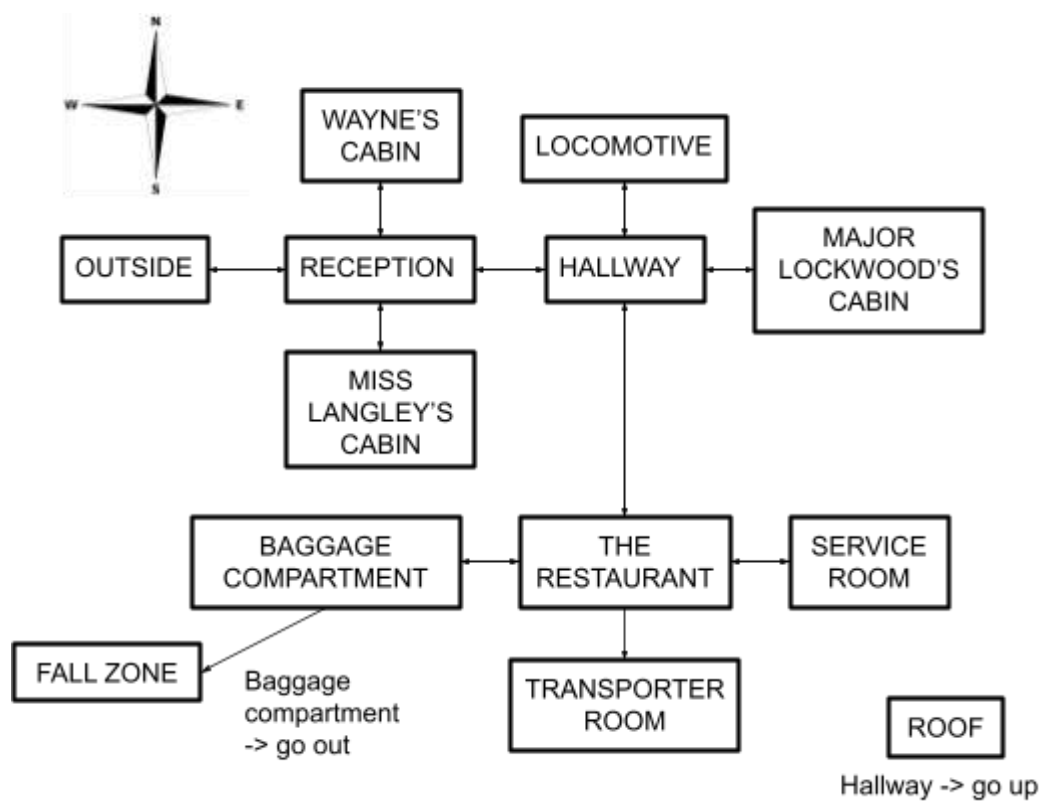
I.B) Thème : En Europe de l'Est, durant l'Entre-Deux-Guerres en 1938, le détective Hercule Poirot doit résoudre le mystère d'un meurtre aux abords du train de l'Orient-Express.

I.C) Résumé du scénario:

Le détective Poirot doit résoudre un mystérieux meurtre à travers des manoeuvres aux abords du train dont l'itinéraire est retardée dû a un objet bloquant les rails (le train est donc immobile pour un certain temps) et ses divers compartiments et chambres. La victime, Thomas Wayne, est en réalité John Cassetti, un criminel recherché pour l'enlèvement et le meurtre d'une enfant. Les passagers, ayant tous un lien avec la victime, ont organisé un

complot de vengeance. Au final, Poirot découvre que le meurtre est un acte collectif de justice.

I.D) Plan:



I.E) Scénario détaillé:

Thomas Wayne est retrouvé assassiné. Poirot commence son enquête en explorant le train et les items qui s'y trouvent. Il découvre que Wayne était en réalité John Cassetti, un criminel responsable du kidnapping et de la mort d'une enfant. Chaque passager a un lien avec la victime et, peu à peu, Poirot va comprendre qu'ils ont tous participé à sa mort pour venger l'enfant.

En fouillant tous les lieux du train et ses diverses chambres comme la cabine de Wayne, le compartiment des bagages et la locomotive, Poirot rassemble des indices cruciaux.

Il finit par résoudre ce mystère qui était en réalité qu'un meurtre en bande organisée. Il a donc résolu le mystère. Les assaillants seront jugés pour leurs crimes une fois le train arrivé à sa destination.

I.F) Détail des lieux, items, personnages:

Personnages

John Cassetti alias Thomas Wayne, Détective Hercule Poirot, Major Lockwood, Miss Langley, le réceptionniste, et le chef du train.

Items

Un stylo-plume: Ce stylo semble tout à fait ordinaire.

Poids : 0,1 kg

Un chronomètre: Ce chronomètre affiche un décalage de temps intrigant.

Poids : 0,2 kg

Un uniforme: Il s'agit d'un uniforme supplémentaire de conducteur de train, bien qu'il n'y en ait généralement qu'un à bord.

Poids : 2 kg

Une montre cassée (1h15): Cette montre s'est arrêtée à 1h15, peut-être l'heure du meurtre.

Poids : 0,3 kg

Un mouchoir brodé: Ce délicat mouchoir porte une seule initiale brodée. À qui peut-il bien appartenir ?

Poids : 0,1 kg

Un verre de vin empoisonné: De légères traces de poison sont visibles.

Était-ce une tentative de meurtre ciblée ?

Poids : 0,4 kg

Un couteau ensanglanté: La lame est tachée de sang. Il a été retrouvé caché derrière un panneau en bois du couloir.

Poids : 1,5 kg

Une clé étiquetée "Wayne" : Cette clé ouvre la cabine de Thomas Wayne.

Pourquoi était-elle cachée ici ?

Poids : 0,1 kg

Un beamer : Cet appareil permet de vous téléporter vers un lieu précédemment visité.

Poids : 2 kg

Une lettre froissée : Elle est adressée au Major Blackwood, sur un ton accusateur.

Poids : 0,2 kg

The doc's elixir : Un élégant flacon noir étiqueté "Élixir du Docteur", réputé pour conférer une force surnaturelle à ceux qui le boivent.

Poids : 0,2 kg

Un billet de train différent : Ce billet indique un lieu d'embarquement différent de celui affirmé par Mlle Langley.

Poids : 0,2 kg

Détail des lieux:

Le train est très luxueux et chic comportant un intérieur en bois de très bonne qualité. L'extérieur du train est bien évidemment en acier, typique des trains de l'époque. À l'extérieur du train, le paysage est enneigé tout autour des rails.

I.G) Situations gagnantes et perdantes:

Situations gagnantes:

- 1) Réunir tous les bons items et accuser les vrais coupables (Major BlackWood, Miss Langley et le réceptionniste). Découverte du meurtre collectif et victoire.

Situations perdantes :

- 1) Dépasser le nombre de déplacements permis (30) en ayant toujours pas résolu le crime.

II. Réponses aux exercices (à partir de l'exercice 7.5 inclus)

Ex 7.5

Pour éviter la duplication de code de l'affichage des sorties de la pièce courante dans `printWelcome()` et `goRoom()`, on crée une méthode `printLocationInfo()` dans la classe `Game` qui auparavant apparaissait dans `printWelcome()` et `goRoom()`.

La création de cette méthode est beaucoup plus efficace en termes de cohésion car elle nous permettra, dans le cas où l'on souhaite modifier le code qui était dupliqué de le faire qu'une seule fois dans la méthode `printLocationInfo()` puis d'appeler cette méthode là où l'on souhaite par *`printLocationInfo()`* au lieu de le modifier autant de fois qu'il apparaissait.

```
private void printLocationInfo()
{
    if (this.aCurrentRoom.aNorthExit != null){
        System.out.print("north ");
    }
    if (this.aCurrentRoom.aSouthExit != null){
        System.out.print("south ");
    }
    if (this.aCurrentRoom.aEastExit != null){
        System.out.print("east ");
    }
    if (this.aCurrentRoom.aWestExit != null){
        System.out.print("west ");
    }
}
} // Game
```

Ex 7.6

Dans cet exercice on écrit un accesseur dans la classe Room qui dépend du String pDirection qu'on lui fournit. Cela permet d'améliorer notre cohésion de code en supprimant 4 tests conditionnels dans la méthode goRoom() de la classe Game, que la méthode getExit() de la classe Room va maintenant incarner en envoyant la pièce concernée lorsqu'elle est appelée.

```
public Room getExit(final String pDirection)
{
    if (pDirection.equals("north")){
        return aNorthExit;
    }
    if (pDirection.equals("south")){
        return aSouthExit;
    }
    if (pDirection.equals("east")){
        return aEastExit;
    }
    if (pDirection.equals("west")){
        return aWestExit;
    }
    return null;
}
} // Room
```

On appelle donc cet accesseur dans goRoom() tel que:

```
String vDirection = pDirection.getSecondWord();
Room vNextRoom = this.aCurrentRoom.getExit(vDirection);
```

Cependant, en écrivant cela, le message "Unknown direction !" n'apparaît plus. Voici comment résoudre ce problème:

Après les tests conditionnels dans getExit(), on retourne l'objet courant. Dans la méthode goRoom() de la classe Game, on vérifie si (vNextRoom == this.aCurrentRoom) soit si la prochaine pièce est la même que la pièce actuelle. Dans ce cas là, on ne change pas de salle et donc la direction est incorrecte: on affiche "Unknown direction !".

```
String vDirection = pDirection.getSecondWord();
Room vNextRoom = this.aCurrentRoom.getExit(vDirection);

if (vNextRoom == this.aCurrentRoom){
    System.out.println("Unknown direction !");
    return;
}
```

Ex 7.7

Nous allons rendre la méthode `printLocationInfo()` plus efficace afin d'améliorer la cohésion de notre code.

On crée une méthode `getExitString()` qui permet d'obtenir les directions possibles à partir d'une pièce.

```
public String getExitString()
{
    String vString="Exits: ";
    if(this.getExit("north") != null){
        vString+="north ";
    }
    if(this.getExit("south") != null){
        vString+="south ";
    }
    if(this.getExit("east") != null){
        vString+="east ";
    }
    if(this.getExit("west") != null){
        vString+="west ";
    }
    return vString;
}
} // Room
```

en modifiant dans `printLocationInfo()`:

```
private void printLocationInfo()
{
    System.out.println("You are currently " +this.aCurrentRoom.getDescription());
    System.out.println(this.aCurrentRoom.getExitString());
}
} // Game
```


Ex 7.8

On crée une hashMap exits qui permettra de stocker aNorthExit, aSouthExit, aEastExit et aWestExit. Cela permettra d'accéder directement à la pièce à partir de sa direction. On change alors le constructeur naturel de la classe Room et l'accesseur getExit() ainsi que la méthode setExits() par une autre méthode setExit() qui positionne les différentes

pièces par rapport à une pièce et une direction. De plus, il y a un changement à faire dans createRooms() dans la classe Game et un autre changement à faire dans getExitString():

```
import java.util.HashMap;
public class Room
{
    private String aDescription; // chaine de caracteres decrivant le lieu
    private HashMap <String, Room> aExits; // direction, chambre dans cette direction

    public Room (final String pDescription) //Constructeur naturel
    {
        this.aDescription= pDescription; // desc. de la piece
        aExits= new HashMap <String, Room>(); //creation d une hashmap
    }

    public String getDescription() // acceseur
    {
        return this.aDescription;
    }

    /**
     * Definir une sortie a paritr de cette piece
     * @param pDirection est la direction dans laquelle on va
     * @param pNeighbor est la chambre voisine, dans cette direction
     */
    public void setExit(final String pDirection, final Room pNeighbor)
    {
        aExits.put(pDirection, pNeighbor);
    }

    /**
     * acceseur de la sortie
     * @param pDirection est la direction pour laquelle on veut connaitre l'issue
     */
    public Room getExit(final String pDirection)
    {
        return aExits.get(pDirection);
    }
}
```



```

public String getExitString()
{
    String vString="Exits: ";
    if(this.aExits.get("north") != null){
        vString+="north ";
    }
    if(this.aExits.get("south") != null){
        vString+="south ";
    }
    if(this.aExits.get("east") != null){
        vString+="east ";
    }
    if(this.aExits.get("west") != null){
        vString+="west ";
    }
    return vString;
}
// Room

```

```

vOutside.setExit("east", vReception);

vReception.setExit("north", vWayneCabin);
vReception.setExit("east", vHallway);
vReception.setExit("west", vOutside);

vWayneCabin.setExit("south", vReception);

vHallway.setExit("north", vLocomotive);
vHallway.setExit("south", vLangleyCabin);
vHallway.setExit("east", vMajorCabin);
vHallway.setExit("west", vReception);

vMajorCabin.setExit("west", vHallway);

vLocomotive.setExit("south", vHallway);

vLangleyCabin.setExit("north", vHallway);
vLangleyCabin.setExit("south", vRestaurant);

vRestaurant.setExit("north", vLangleyCabin);
vRestaurant.setExit("east", vServiceRoom);
vRestaurant.setExit("west", vBaggage);

vServiceRoom.setExit("west", vRestaurant);
vBaggage.setExit("east", vRestaurant);

```

Après un test, on remarque que le message "Unknown Direction !" ne s'affiche pas: j'ai donc résolu ce problème avec le code qui suit:

ajout de ce code dans CommandWords:

```
private static final String[] aValidDirections = {"north", "south", "east", "west"};
```

```
/**
 * Check whether a given String is a valid command word. |
 * @return true if a given string is a valid command,
 * false if it isn't.
 */
public static boolean isDirections( final String pString )
{
    for ( int vI=0; vI<aValidDirections.length; vI++ ) {
        if (aValidDirections[vI].equals( pString ) )
            return true;
    } // for
    // if we get here, the string was not found in the commands :
    return false;
} // isDirections()
```

(du même style que isCommand).

Ex 7.8.1

Ajout d'une "pièce" en hauteur: le toit du train, pour inspecter s'il y a eu des intrusions/ des trappes ouvertes.

Exercices 7.9 et 7.10 Faits

la méthode getExitString() mise à jour:

```
/**
 * Retourne une description des différentes sorties,
 * Par exemple: "Exits: north south"
 */
public String getExitString()
{
    String vString="Exits: ";
    Set <String> vKeys = aExits.keySet();
    for(String vExit : vKeys) {
        vString+= " " + vExit;
    }
    return vString;
}

} // Room
```

Exercices 7.10.1 & 7.10.2:

Javadoc complété et généré.

Ex 7.11

Ajout de la méthode `getLongDescription()` afin d'améliorer la cohésion du code:

```
/**
 * Methode retournant une description de la piece actuelle
 * avec les exits actuelles.
 */
public String getLongDescription(){
    return "You are " + aDescription + ".\n" + getExitString();
}
Room
```

elle sera appelée par la suite dans `printLocationInfo()` tel que:

```
private void printLocationInfo()
{
    System.out.println(this.aCurrentRoom.getLongDescription());
} // printLocationInfo
} // Game
```

Ex 7.14

```
private final String[] aValidCommands = { "go", "help", "quit", "look" };
```

```
private void look()
{
    System.out.println(this.aCurrentRoom.getLongDescription());
}
```

```

private boolean processCommand(final Command pCommandProcess){
    if (pCommandProcess.isUnknown()){
        System.out.println("I don't know what you mean...");
        return false;
    }
    else if(pCommandProcess.getCommandWord().equals("quit")){
        return (this.quit(pCommandProcess));
    }
    else if(pCommandProcess.getCommandWord().equals("help")){
        this.printHelp();
        return false;
    }
    else if(pCommandProcess.getCommandWord().equals("go")){
        this.goRoom(pCommandProcess);
        return false;
    }
    else if(pCommandProcess.getCommandWord().equals("look")){
        this.look();
        return false;
    }
    System.out.println("Erreur du programmeur: commande non reconnue!");
    return true;
}

```

Ex 7.15

```

private void eat(){
    System.out.println("You have eaten now and you aren't hungry anymore");
}

private final String[] aValidCommands = { "go", "help", "quit", "look", "eat"};

else if(pCommandProcess.getCommandWord().equals("eat")){
    this.eat();
    return false;
}

```

Ex 7.16

Dans Parser:


```

/**
 * Prints out a list of all valid command words
 */
public void showCommands(){
    this.aValidCommands.showAll();
}

```

Dans Commandwords:

```

/**
 * Print all valid commands
 */
public void showAll(){
    for(String vCommand: aValidCommands){
        System.out.print(vCommand+" ");
    }
    System.out.println();
}

```

dans game :

```

private void printHelp(){
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around the train's premises.");
    System.out.println();
    System.out.println("Your command words are:");
    aParser.showCommands();
}

```

Ex 7.18

Dans parser:

```

/**
 * Prints out a list of all valid command words
 */
public String getCommands(){
    return aValidCommands.getCommandList();
} // getCommands

```

Dans Game:

```
System.out.println(aParser.getCommands());
```

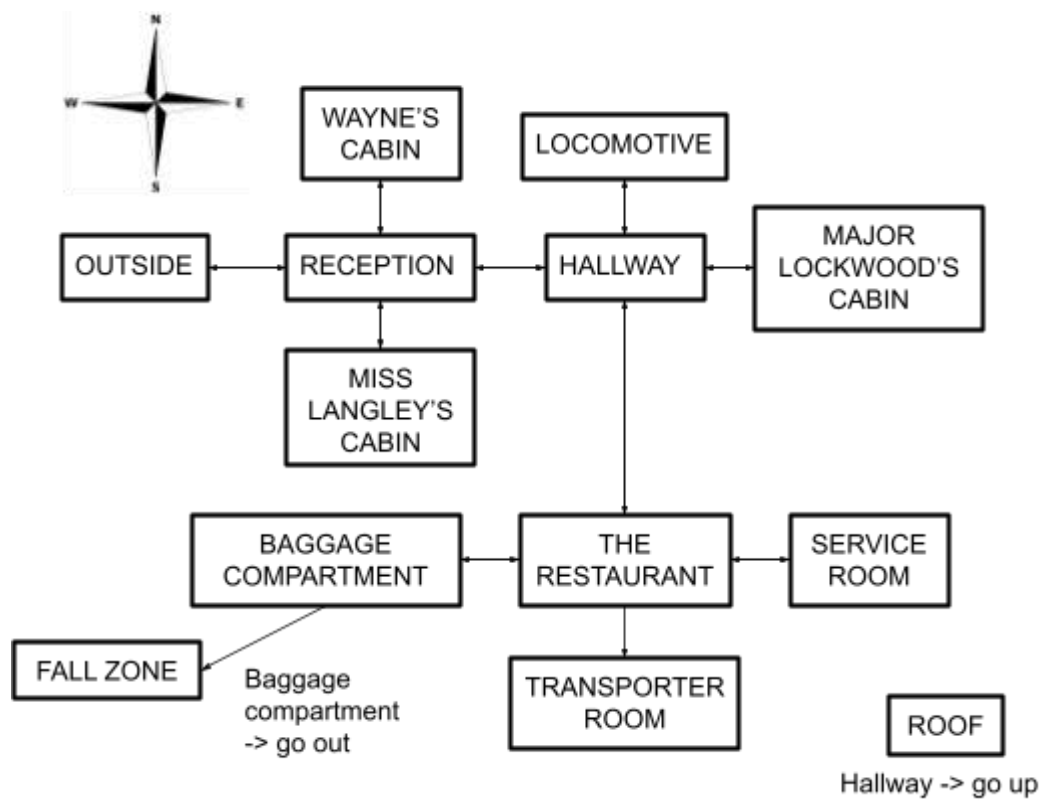
Dans CommandWords:

```
/**
 * Print all valid commands
 */
public String getCommandList(){
    String vString= "";
    for(String vCommand:aValidCommands){
        vString+=vCommand+" ";
    }
    return vString;
}
```

Ex 7.18.1

Comparaison avec zuul better. Pas de modifications à effectuer.

Ex 7.18.3



Voici les images choisies pour les différentes pièces:

Outside



Reception



Wayne's cabin



Hallway



Major Lockwood's Cabin



Locomotive



Miss Langley's Cabin



The restaurant



Service Room



Baggage Compartment



Fall Zone



Transporter Room



Roof



Ex 7.18.4

Titre du jeu: All Aboard For Murder

Ex 7.18.6

Nous effectuons quelques modifications:

ajout de l'attribut `aImageName` , et de son initialisation dans le constructeur:

```
public Room (final String pDescription, final String pImageName)
{
    | this.aDescription= pDescription; // description de la piece
    aExits= new HashMap <String, Room>();
    this.aImageName=pImageName;
}
```

Nous écrivons aussi la méthode suivante:

```
public String getImageName() {
    return this.aImageName;
}
```

On modifie le Parser également; on utilise maintenant non pas un scanner mais un `StringTokenizer`.

```
import java.util.StringTokenizer;
```

```
/**
 * This class is part of the "World of Malice" application.
 * "World of Malice" is a very simple, text based adventure game.
 *
 * This parser reads user input and tries to interpret it as an "Adventure"
 * command. Every time it is called it reads a line from the terminal and
 * tries to interpret the line as a two word command. It returns the command
 * as an object of class Command.
 *
 * The parser has a set of known command words. It checks user input against
 * the known commands, and if the input is not one of the known commands, it
 * returns a command object that is marked as an unknown command.
 *
 * @author Michael Kolling and David J. Barnes + D.Bureau + Youssef Shalaby
 * @version 2008.03.30 + 2013.09.15 + 14/02/25
 */
```

```
public class Parser
```

```
{
```

```
    private CommandWords aValidCommands; // (voir la classe CommandWords)
```

```
    /**
     * Constructeur par default qui cree les 2 objets prevus pour les attributs
     */
```

```
    public Parser()
```

```
    {
```

```
        this.aValidCommands = new CommandWords();
```

```
    } // Parser()
```

```

public Command getCommand(final String pInputLine)
{
    String vWord1;
    String vWord2;

    StringTokenizer tokenizer = new StringTokenizer( pInputLine );

    if ( tokenizer.hasMoreTokens() )
        vWord1 = tokenizer.nextToken();    // get first word
    else
        vWord1 = null;

    if ( tokenizer.hasMoreTokens() )
        vWord2 = tokenizer.nextToken();    // get second word
    else
        vWord2 = null;

    // note: we just ignore the rest of the input line.

    // Now check whether this word is known. If so, create a command
    // with it. If not, create a "null" command (for unknown command).

    if ( this.aValidCommands.isCommand( vWord1 ) ) {
        return new Command( vWord1, vWord2 );
    }
    else {
        return new Command( null, vWord2 );
    }
} // getCommand(.)

/**
 * Returns a String containing all valid command words
 */
public String getCommandString(){
    return this.aValidCommands.getCommandList();
} // getCommands

```

Nous effectuons les modifications suivantes dans la classe GameEngine, avec l'ajout de la méthode setGUI.

```
private UserInterface aGui; // 1 interface
```

```
public GameEngine(){ // constructeur par défaut
    this.createRooms();
    this.aParser= new Parser();
}
```

```
public void setGUI( final UserInterface pUserInterface )
{
    this.aGui = pUserInterface;
    this.printWelcome();
}
```

Dans GameEngine (précédemment Game) , on transforme également processCommand() en interpretCommand().

```
private void interpretCommand(final String pCommandLine){
    this.aGui.println("> " + pCommandLine);
    Command vCommand = this.aParser.getCommand(pCommandLine);

    if (vCommand.isUnknown()){
        this.aGui.println("I don't know what you mean...");
        return;
    }
    String vCommandWord = vCommand.getCommandWord();

    if ( vCommandWord.equals( "help" ) )
        this.printHelp();
    else if ( vCommandWord.equals( "go" ) )
        this.goRoom( vCommand );
    else if ( vCommandWord.equals( "look" ) )
        this.look();
    else if ( vCommandWord.equals( "eat" ) )
        this.eat();
    else if ( vCommandWord.equals( "quit" ) ) {
        if ( vCommand.hasSecondWord() )
            this.aGui.println( "Quit what?" );
        else
            this.endGame();
    }
}
```

Nous remplaçons ensuite la méthode quit par la méthode EndGame, qui sera appelée dans interpretCommand dans le cas où le joueur veut mettre fin au jeu.

```
/**
 * Pour quitter le jeu
 */
private void endGame(){
    this.aGui.println( "Thank you for playing, Good bye!" );
    this.aGui.enable( false );
} // endGame()
```

Dans la classe UserInterface, afin d'uniquement importer les classes nécessaires, nous effectuons les changements suivants.

```
//import javax.swing.*;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JTextArea;
import javax.swing.JLabel;
import javax.swing.ImageIcon;
import javax.swing.JScrollPane;
import javax.swing.JPanel;

//import java.awt.*;
import java.awt.Dimension;
import java.awt.BorderLayout;

//import java.awt.event.*;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import java.net.URL;
//import java.awt.image.*;
import java.awt.event.ActionEvent;
```


Ex 7.18.8

Pour ajouter un bouton, on importe tout d'abord la class JButton dans la classe UserInterface.

```
import javax.swing.JButton;
```

On va créer un attribut pour chaque bouton: les boutons que j'ai choisi d'implémenter sont les suivants:

```
private JButton aEastButton;  
private JButton aUpButton;  
private JButton aWestButton;  
private JButton aNorthButton;  
private JButton aSouthButton;  
private JButton aOutButton;  
  
private JButton aInventoryButton;  
private JButton aLookButton;  
private JButton aQuitButton;  
private JButton aHelpButton;
```

Dans la méthode createGUI, on écrit le suivant, afin d' initialiser les boutons:

```
this.aEastButton = new JButton("Go East");  
this.aWestButton = new JButton("Go West");  
this.aNorthButton = new JButton("Go North");  
this.aSouthButton = new JButton("Go South");  
this.aOutButton = new JButton("Go Out");  
this.aUpButton = new JButton("Go Up");  
  
this.aInventoryButton = new JButton("See Inventory");  
this.aLookButton = new JButton("Look Around");  
this.aQuitButton = new JButton("Quit");  
this.aHelpButton = new JButton("Help Menu");
```

Nous avons donc créé un panel de boutons, et ces boutons sont visibles dans le jeu avec les commandes suivantes:


```

vButtonPanel.add(this.aNorthButton);
vButtonPanel.add(this.aSouthButton);
vButtonPanel.add(this.aEastButton);
vButtonPanel.add(this.aWestButton);
vButtonPanel.add(this.aUpButton);
vButtonPanel.add(this.aOutButton);

vButtonPanel.add(this.aInventoryButton);
vButtonPanel.add(this.aLookButton);
vButtonPanel.add(this.aQuitButton);
vButtonPanel.add(this.aHelpButton);

```

Pour rendre le panel de boutons fonctionnel, on commence par rajouter un ActionListener pour chaque bouton.

```

this.aEastButton.addActionListener(this);
this.aWestButton.addActionListener(this);
this.aNorthButton.addActionListener(this);
this.aSouthButton.addActionListener(this);
this.aUpButton.addActionListener(this);
this.aOutButton.addActionListener(this);

this.aInventoryButton.addActionListener(this);
this.aLookButton.addActionListener(this);
this.aQuitButton.addActionListener(this);
this.aHelpButton.addActionListener(this);

```

Les boutons s'affichent bien comme il faut, et sont fonctionnels:



Avec l'aide du site [Java AWT | GridLayout Class | GeeksforGeeks](https://www.geeksforgeeks.org/java-awt-gridlayout-class/), j'ai aussi programmé un Grid 4x4:

```

JPanel vButtonPanel = new JPanel(new GridLayout(4,4));

```

Ex 7.19.2

En mettant toutes les images dans un repertoire “ Images”,

on doit effectuer la modification suivante

```
v0Outside = new Room("outside the train, by the tracks.", "Images/v0Outside.jpg");
```

Ex 7.20

Nous créons la classe Item, et donc nous ajoutons un attribut Item dans Room afin qu'il y ait un item dans une room. Nous créons les attributs suivants pour Item afin de faciliter leur utilisation: name(leur nom), Description (une brève description) et Poids(leur poids).

On écrit un getter **getItem** afin d'obtenir l'item, et un setter **setItem** afin d'associer l'item au lieu dans lequel il se trouve.

```
public class Item
{
    private String aDescription;
    private String aName;
    private double aPoids;

    /**
     * @param pDescription description of the item
     * @param pName name of the item
     * @param pPoids weight of the item
     */
    public Item(final String pDescription, final String pName, final double pPoids)
    {
        // initialisation des variables d'instance
        this.aDescription = pDescription;
        this.aName = pName;
        this.aPoids = pPoids;
    }

    /**
     * Getter de la description
     * @return la description de l'item
     */
    public String getDescription(){
        return this.aDescription;
    }
}
```

```

/**
 * Getter du poids
 * @return le poids de l'item
 */
public double getPoids(){
    return this.aPoids;
}

```

```

/**
 * Getter du nom de l'item
 * @return le nom de l'item
 */
public String getName(){
    return this.aName;
}

```

dans la classe Room:

```
private Item aItem;
```

```

/**
 * procedure pour associer l'item au lieu
 */
public void setItem(final Item pItem){
    this.aItem=pItem;
}

```

```

/**
 * accesseur de l'item
 */
public Item getItem(){
    return this.aItem;
}

```

+ Modification de la méthode get long description dans room:

```

/**
 * Methode retournant une description de la piece actuelle
 * avec les exits actuelles.
 */
public String getLongDescription(){
    String vDesc= "You are " + aDescription + ".\n" + getExitString();
    if(this.aItem!= null){
        vDesc = vDesc + "There is " + this.aItem.getName() + '\n' + "this item weighs " + this.aItem.getPoids();
    }
    else {
        vDesc = vDesc + "There are no items here.";
    }
    return vDesc;
}

```

Ex 7.21

ajout de la description de l'item dans la classe Item.

```
public String getItemDescription(){  
    return "There is " + this.aDescription + "this item weighs " + this.aPoids;  
}
```

Ex 7.22

Nous souhaitons modifier le projet afin qu'une Room puisse contenir plusieurs objets. Pour cela, on ajoute une procédure addItem qui va placer un item dans Room.

D'abord, on ajoute une Hashmap aItems dans les attributs de Room:

```
private HashMap<String, Item> aItems;
```

```
/**  
 * cree un item pour une room  
 * @param pItem  
 * @param pName  
 */  
public void addItem(final String pName, final Item pItem){  
    this.aItems.put(pName, pItem);  
} // addItem()
```

```
/**  
 * accesseur de l'item ou des items  
 */  
public Item getItem(final String pNom){  
    return this.aItems.get(pNom);  
}
```

Il faut également créer une fonction getItemString qui nous permettra d'afficher le nom de l'item:

```

public String getItemString (){
    StringBuilder returnString = new StringBuilder( "Items:" );
    Set<String> vObjects = aItems.keySet();//vObjects sont les n
    for (String vKey:vObjects){
        returnString.append( " " + vKey);
    }
    return returnString.toString();
}

```

Ex 7.22.2

Voici les items.

```

//Items
vLocomotive.setItem(new Item("a stopwatch.", "This stopwatch presents a time difference", 0.2));
vBaggage.setItem(new Item("a uniform.", "This is an extra train driver's uniform, although there is usually only one on board.", 2));
vWayneCabin.setItem(new Item("a broken watch (1:15).", "This watch stopped at 1:15, possibly indicating the time of the murder.", 0.3));
vReception.setItem(new Item("an embroidered handkerchief.", "This delicate handkerchief has a single embroidered initial. Whose is it?", 0.1));
vRestaurant.setItem(new Item("a poisoned wine glass.", "There are faint traces of poison in this wine glass. Was someone targeted?", 0.4));
vHallway.setItem(new Item("a bloody knife.", "The blade is stained with blood. It was found tucked away behind a panel.", 1.5));
vServiceRoom.setItem(new Item("a key labeled 'Wayne'.", "This key opens Thomas Wayne's cabin. Why was it hidden here?", 0.1));
vMajorCabin.setItem(new Item("a crumpled letter.", "The letter is addressed to Major Blackwood, and its tone is accusatory.", 0.2));
vLangleyCabin.setItem(new Item("a different train ticket.", "This ticket has a different departure location than Miss Langley's claimed boarding point.", 0.2));

```

Ex 7.23

Nous devons implémenter une commande back qui permettra au joueur de revenir dans la pièce dans laquelle il était.

Pour cela, il faut tout d'abord ajouter la commande "back" comme commande valide, et cela dans l'attribut aValidCommands de la classe CommandWords:

```

private static final String[] aValidCommands = { "go", "back", "help", "quit", "look", "eat"}

```

Afin d'accéder à la pièce précédente, on ajoute cet attribut dans Game Engine, et on l'initialise dans goRoom():

```

private Room aPreviousRoom;

this.aPreviousRoom=null;

```

A présent, on crée la méthode back():


```

**
 * Pour revenir en arriere
 *
 * @param pCommand commande du joueur
 */
private void back(final Command pCommand){
    // si presence de deuxieme mot
    if (pCommand.hasSecondWord()){
        this.aGui.println("Go back where?");
        return;
    }// if
    // cas ou il n y a pas de piece precedente
    Room vNextRoom = this.aPreviousRoom;
    if (vNextRoom==null){
        this.aGui.println("There isn't any previous room");
        return;
    }
    this.aPreviousRoom=this.aCurrentRoom;
    this.aCurrentRoom=vNextRoom;
    this.aGui.showImage(this.aCurrentRoom.getImageName());
    printLocationInfo();
} // back ()

```

Il faut également ajouter le suivant dans la procédure interpretCommand():

```

else if ( vCommandWord.equals( "back" ) )
    this.back(vCommand);

```

Ex 7.24

Exercice complété

Ex 7.25

Exercice complété.

Ex 7.26

Nous remarquons qu'un des problèmes de back() est le fait que cliquer back deux fois nous ramène à la salle du début. On implémente donc Stack. On importe Stack d'abord dans GameEngine.

```

import java.util.Stack;

```


on ajoute ça comme attribut:

```
private Stack <Room> aPreviousRooms;
```

et on l'initialise:

```
public GameEngine(){ // constructeur par défaut
    this.createRooms();
    this.aParser= new Parser();
    this.aPreviousRooms= new Stack<Room>();
}
```

on modifie back:

```
/**
 * Pour revenir en arriere
 *
 * @param pCommand commande du joueur
 */
private void back(){
    // si presence de deuxieme mot
    if (this.aPreviousRooms.empty()){
        this.aGui.println("There isn't any previous room");
    }
    else{
        this.aCurrentRoom=this.aPreviousRooms.pop();
    }
    this.aGui.showImage(this.aCurrentRoom.getImageName());
    printLocationInfo();
} // back ()
```

Maintenant, on peut faire back plusieurs fois sans problème.

Ex 7.26.1

Javadoc.

Ex 7.27

J'ai lu et compris l'idée de refactoring et testing.

Ex 7.28

Exercice fait.

Ex 7.28

Dans GameEngine, nous créons une nouvelle commande test représentant le nom de fichier. Cette méthode test sera comme suit:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
```

on ajoute d'abord le mot "test" dans la liste des commandwords.

```
if ( vCommandWord.equals( "test" ) )
    this.test(vCommand);
```

ça dans interpretCommand() de la classe GameEngine.

J'utilise les instructions java try/catch qui sont très utiles ici.

- Méthode test:

```
private void test(final Command pInput)
{
    //test absence du second mot
    if (!pInput.hasSecondWord()){
        this.aGui.println("Test which file?");
        return;
    }

    String vFileName = pInput.getSecondWord();
    vFileName=vFileName + ".txt";
    //si presence du second mot
    String vFile = pInput.getSecondWord();
    try { // teste le fichier
        Scanner vScan= new Scanner(new File(""+ vFile + ".txt"));
        this.aGui.println("Now testing " + vFile + ".");
        while (vScan.hasNextLine()){
            interpretCommand(vScan.nextLine());
        }
    } catch (final FileNotFoundException pFileNotFoundException) { // catch en cas d'absence du fichier
        this.aGui.println("Such file does not exist.");
    } // final: pFileNotFoundException ne peut subir aucune modification
}
```

Ex 7.28.2

Création des fichiers court.txt, ideal.txt, et possibilities.txt

Ex 7.29

La classe GameEngine devenant trop grande et susceptible de causer des erreurs, l'idée de refactoring nous indique qu'il faudrait créer une classe player qui contiendrait certaines méthodes de la classe gameEngine, et des informations sur le player.

On stockera des informations telles que la pièce dans laquelle le player est actuellement, et les précédentes pièces dans lesquelles il était.

Voici à quoi ressemble la classe Player avec les changements nécessaires:

```
import java.util.Stack;

/**
 * Classe Player
 *
 * @author Youssef Shalaby
 * @version -
 */
public class Player
{
    private String aPseudonym; // pseudo au cas ou il ya plusieurs players
    private Room aCurrentRoom; // piece actuelle
    private Stack <Room> aPreviousRooms; // precedente piece

    /**
     * Constructeur naturel
     *
     * @param pPseudonym le pseudonyme du joueur
     * @param pCurrentRoom piece de depart
     */
    public Player(final String pPseudonym, final Room pCurrentRoom){
        this.aPseudonym = pPseudonym;
        this.aCurrentRoom = pCurrentRoom;
        this.aPreviousRooms = new Stack <Room> ();
    } // Player()
}
```

```

/**
 * Getter du pseudonyme
 *
 * @return Pseudonyme du joueur
 */
public String getPseudonym()
{
    return this.aPseudonym;
} // getPseudo()

/**
 * Getter de la piece actuelle du joueur
 *
 * @return Piece actuelle du joueur
 */
public Room getCurrentRoom()
{
    return this.aCurrentRoom;
} // getCurrentRoom()

/**
 * Getter du stack des precedentes pieces
 *
 * @return Stack des precedentes pieces
 */
public Stack <Room> getPreviousRooms()
{
    return this.aPreviousRooms;
} // getPreviousRooms()

```

```

/**
 * Pour le deplacement entre les pieces
 *
 * @param pStringRoom Direction de la piece a recuperer
 */
public void goRoom(final String pRoom)
{
    Room vNextRoom = this.aCurrentRoom.getExit(pRoom); //
    this.aPreviousRooms.push(this.aCurrentRoom); //Modifi
    this.aCurrentRoom = vNextRoom; //changement de la pie
} // goRoom()

/**
 * Afin de revenir en arriere
 */
public void back()
{
    this.aCurrentRoom = this.aPreviousRooms.pop(); //Char
} // back()
} // Player

```

Dans GameEngine, on ajoute un attribut aPlayer, et on l'initialise comme suit dans createRooms():

```
this.aPlayer = new Player ("Youssef", vOutside);
```

la méthode goRoom devient la suivante:

```
/**
 * goRoom permet de se deplacer aux abords du train.
 * @param pInput nous indique la direction dans laquelle on se deplace
 */
private void goRoom (final Command pInput){
    if(!pInput.hasSecondWord()){
        this.aGui.println("Go where ?");
        return;
    } // si il n y a pas de deuxieme mot on affiche "Go Where?"

    String vDirection = pInput.getSecondWord();

    if (this.aPlayer.getCurrentRoom().getExit(vDirection) == null ){
        this.aGui.println( "There is no door!" );
    } // end if

    this.aPlayer.goRoom(vDirection);
    printLocationInfo();
    if (this.aPlayer.getCurrentRoom().getImageName() != null){
        this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName());
    }
}

/goRoom
```

et back() dans GameEngine:


```

/**
 * Pour revenir en arriere
 *
 * @param pCommand commande du joueur
 */
private void back(){
    // si presence de deuxieme mot
    //if (this.aPreviousRooms.empty())
    if (this.aPlayer.getPreviousRooms().empty()){
        this.aGui.println("There isn't any previous room.");
    }
    else{
        this.aPlayer.back();
    }
    this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName());
    printLocationInfo();
} // back ()

```

Comme vu ci-dessus, aPreviousRoom et aCurrentRoom ont respectivement été modifiées par “aPlayer.getPreviousRoom()” et “aPlayer.getCurrentRoom()”. dans la méthode back, ça devient this.aPlayer.Back(), qui vient remplacer la ligne “this.aCurrentRoom = this.aPreviousRooms.pop();”.

Ex 7.30

On ajoute take et drop à la liste des commandes.

On ajoute ensuite un attribut altem dans Player.

- On ajoute un setter setItem pour ajouter un Item au joueur.
- Dans la classe Room, on ajoute une procédure RemoveItem pour enlever un item d' une pièce.

```

public void removeItem(final String pName){
    this.aItems.remove(pName);
} // removeItem()

```

- on ajoute dans GameEngine les méthodes take et drop, ainsi qu'un getter getItem dans Player.

```

private void take (final Command pInput){
    if(!pInput.hasSecondWord()){
        this.aGui.println("Take what?");
        return;
    }

    String vItemName = pInput.getSecondWord();
    Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);
    if(vItem == null){
        this.aGui.println("there's no such item here !");
        return;
    }
    this.aPlayer.setItem(vItem);
    this.aPlayer.getCurrentRoom().removeItem(vItemName);
} //take()

```

```

private void drop (final Command pInput){
    if(!pInput.hasSecondWord()){
        this.aGui.println("Drop what?");
        return;
    }

    String vItemName = pInput.getSecondWord();
    Item vItem = this.aPlayer.getItem();
    if(vItem == null){
        this.aGui.println("there's no such item here !");
        return;
    }

    this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
    this.aPlayer.removeItem();
}

```

Ex 7.31

Afin de porter plusieurs items, on fait appel à une HashMap des items que l'on initialise dans Player. On modifie donc ce qui auparavant marchait que pour un seul item: Dans Player on a donc:

```

public void addItem(final String pName, final Item pItem) {
    this.aItems.put(pName, pItem);
}

public Item getItem(final String pName){
    return this.aItems.get(pName);
} // getItem()

public Item removeItem(final String pName){
    return this.aItems.remove(pName);
} // removeItem()

```

Ex 7.31.1

```

import java.util.HashMap;
import java.util.Set;

```

```

public class ItemList
{
    private HashMap<String, Item> aItems;

    public ItemList(){
        this.aItems = new HashMap<String, Item>();
    }

    public void addItem(final String pName, final Item pItem){
        this.aItems.put(pName, pItem);
    }

    public Item getItem(final String pName){
        return this.aItems.get(pName);
    } // getItem()

    public Set<String> getKeys(){
        return this.aItems.keySet();
    }

    public Item removeItem(final String pName){
        return this.aItems.remove(pName);
    } // removeItem()
}

```

Ex 7.32

On crée les attributs aWeight et aMaxWeight dans Player.

on crée un getter getWeight() et un setter setWeight().

on modifie les méthodes take() et drop() pour qu'elles prennent en compte le poids des items.

```
private void take (final Command pInput){
    String vItemName = pInput.getRestOfLine();
    if(vItemName==null || vItemName.isEmpty()){
        this.aGui.println("Take what?");
        return;
    }

    Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);
    if(vItem == null){
        this.aGui.println("there's no such item here !");
        return;
    }

    double vCurrentWeight = this.aPlayer.getWeight();
    double vItemWeight = vItem.getWeight();
    double vMaxWeight = this.aPlayer.getMaxWeight();

    if (vCurrentWeight + vItemWeight > vMaxWeight) {
        this.aGui.println("You can't carry that much weight.");
        return;
    }

    this.aPlayer.addItem(vItemName, vItem);
    this.aPlayer.setWeight(vCurrentWeight + vItemWeight);
    this.aPlayer.getCurrentRoom().removeItem(vItemName);
    this.aGui.println("You picked up the " + vItemName + ".");
} //take()
```



```

private void drop (final Command pInput){
    if(!pInput.hasSecondWord()){
        this.aGui.println("Drop what?");
        return;
    }

    String vItemName = pInput.getSecondWord();
    Item vItem = this.aPlayer.getItem(vItemName);

    double vCurrentWeight = this.aPlayer.getWeight();
    double vItemWeight = vItem.getWeight();
    if(vItem == null){
        this.aGui.println("you don't have that item !");
        return;
    }
    this.aPlayer.setWeight(vCurrentWeight - vItemWeight);
    this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
    this.aPlayer.removeItem(vItemName);
    this.aGui.println("You dropped the " + vItemName + ".");
} //drop()

```

Ex 7.33

On crée la méthode Inventory dans GameEngine, qui s'occupe d'afficher tous les détails sur les items dans l'inventaire actuel.

```

private void inventory(){
    ItemList vItems = this.aPlayer.getInventory();

    if (vItems.isEmpty()) {
        this.aGui.println("You are not carrying anything.");
    } else {
        this.aGui.println("You are carrying:");
        double vTotalWeight = 0;
        for (String vItemName : vItems.getKeys()) {
            Item vItem = vItems.getItem(vItemName);
            double vItemWeight = vItem.getWeight();
            vTotalWeight+= vItemWeight;
            this.aGui.println("- " + vItemName + ": " + vItem.getDescription() + " (Weight: " + vItemWeight+ " Kg)" );
        }
        this.aGui.println("You are carrying " + vTotalWeight + " kg " + "worth of items.");
    }
} // inventory()

```

Ex 7.34

l'item sera nommé "The Doc's Elixir."

on enlève le final de l'attribut aMaxWeight pour qu'il soit modifiable.

on ajoute la commande "drink" à la liste de commandes valides,

ajout de cette méthode dans Player:

```
public void DoubleMaxWeight(){  
    this.aMaxWeight = 10;  
}
```

et la méthode drink:

```
private void drink (final Command pInput){  
    String vItemName = pInput.getRestOfLine();  
    if(!pInput.hasSecondWord()){  
        this.aGui.println("Drink what?");  
        return;  
    }  
  
    Item vItem = this.aPlayer.getItem(vItemName);  
    if(vItem == null){  
        this.aGui.println("There is no such drink !");  
        return;  
    }  
  
    if(!vItemName.equals("doc's elixir")){  
        this.aGui.println("You can't drink that.");  
        return;  
    }  
  
    double vCurrentWeight = this.aPlayer.getWeight();  
    double vItemWeight = vItem.getWeight();  
    double vMaxWeight = this.aPlayer.getMaxWeight();  
  
    this.aPlayer.doubleMaxWeight();  
    this.aPlayer.removeItem(vItemName);  
    this.aPlayer.setWeight(vCurrentWeight - vItemWeight);  
    this.aGui.println("You drank the " + vItemName + ". Your maximum load has been increased to " + vMaxWeight*2 + " kg.");  
} //drink
```

Ex 7.34.1 et 7.34.2

Exercices complets.

Ex 7.42

La limite que je vais ajouter a mon jeu sera une limite sur le nombre de déplacements (ex. go west, go east) que le joueur peut faire.

on ajoute dans player l'attribut:

```
private int aMoves;
```

et ensuite on crée la méthode addMove qui va répertorier le nombre de moves avec le getter getNbMoves:

```

public void addMoves(){
    this.aMoves++; // ajoute 1 au moves
}

public int getMoves(){
    return this.aMoves;
}

```

Dans GameEngine:

on ajoute (30 est un exemple);

```

private final int aMaxMoves=30;

```

Puis ajout de la méthode lose au cas ou on dépasse le nombre maximal de déplacements:

```

private void lose(){
    this.aGui.println( "You lost the game. You moved around too much for no reason." );
    this.endGame();
}

```

finalement, on modifie goRoom pour prendre en compte ce cas la:

```

private void goRoom (final Command pInput){
    if(this.aPlayer.getMoves() >= this.aMaxMoves){
        this.lose();
        return;
    }

    if(!pInput.hasSecondWord()){
        this.aGui.println("Go where ?");
        return;
    } // si il n y a pas de deuxieme mot on affiche "Go Where?"

    String vDirection = pInput.getSecondWord();

    if (this.aPlayer.getCurrentRoom().getExit(vDirection) == null ){
        this.aGui.println( "There is no door!" );
        return;
    } // end if

    this.aPlayer.goRoom(vDirection);
    this.aPlayer.addMove();
    printLocationInfo();
}

```

J'ai voulu également afficher le nombre de déplacements qui restent après chaque fois que l'on fait un déplacement, j'ai donc ajouté ces deux lignes dans goRoom:

```

int vMovesLeft = this.aMaxMoves - this.aPlayer.getMoves();
this.aGui.println("You have " + vMovesLeft + " moves left.");

```

Ex 7.42.2

Je souhaite me contenter de l'interface actuelle.

Ex 7.43

J'ajoute ici le toit (vRoof) et donc j'ajoute la commande up et j'ajoute ces lignes dans GameEngine:

```

vRoof = new Room("on the train's roof", "Images/vRoof.jpg");

vHallway.setExit("up", vRoof);

```

Nous commençons par créer une méthode nommée `isExit` dans la classe `Room` qui va déterminer si la pièce dans laquelle on était précédemment peut être considérée comme sortie de la pièce dans laquelle on est actuellement.

Ici, la première trapdoor sera donc celle qui mène au toit.

```
public boolean isExit(final Room pRoom){  
    if (this.aExits.containsValue(pRoom)){  
        return true;  
    }  
    return false;  
}
```

et on change le `back()` de `GameEngine`:

```
private void back() {  
    if (this.aPlayer.getPreviousRooms().empty()) {  
        this.aGui.println("There isn't any previous room.");  
    } else {  
        Room vPreviousRoom= this.aPlayer.getPreviousRooms().peek();  
        Room vCurrentRoom= this.aPlayer.getCurrentRoom();  
  
        if (vCurrentRoom.isExit(vPreviousRoom)) {  
            this.aPlayer.back();  
        } else {  
            this.aGui.println("Can't go back that way !");  
        }  
    }  
    this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName());  
    printLocationInfo();  
} //back()
```

(on utilise `peek` pour regarder la dernière pièce sur la pile sans la retirer.)

Cependant, on remarque une légère inconvenience dans l'affichage: sur le toit, il s'affiche: "You can go in the following directions:" et aucune direction après les deux points. Pour résoudre ce problème, on modifie `getExitString()` dans `Room`: on ajoute le cas où il n'y a aucune sortie.

```
if (this.aExits.isEmpty()){  
    return ("It seems like there are no exits...");  
}
```

et ça marche.

Pour implémenter une deuxième trapdoor, c' est simple, il faut juste ajouter une ligne qui est:

```
vBaggage.setExit("out", vFallZone);
```

et donc dans le cas ou on tape "go out" dans vBaggage, on quitte le train, atterrissant dans vFallZone, et on ne peut plus remonter dans le train.

- il faut donc ajouter "out" à la liste des directions dans CommandWords.

Ex 7.44

On crée une classe Beamer qui hérite de Item.

- ajout de l'item "Beamer", qui sera dans vServiceRoom. (attention, il faut écrire new Beamer et non new item)
- On ajoute les commandes du beamer "charge" et "fire" dans aValidCommands.

voici la classe Beamer:


```

public class Beamer extends Item
{
    private Room aChargedRoom; // piece ou le beamer a ete charge
    /**
     * Constructeur naturel
     *
     * @param pName Beamer's name
     * @param pDescription Beamer's description
     * @param pWeight Beamer's weight
     */
    public Beamer(final String pName, final String pDescription, final double pWeight)
    {
        super(pName, pDescription, pWeight);
    } // Beamer()

    /**
     * Accesseur de la piece chargee
     *
     * @return Charged room
     */
    public Room getChargedRoom()
    {
        return this.aChargedRoom;
    } // getChargedRoom()

```

```

    /**
     * Modificateur de la piece chargee
     *
     * @param pRoom piece dans laquelle on charge le beamer
     */
    public void setChargedRoom(final Room pRoom)
    {
        this.aChargedRoom = pRoom;
    } // setChargedRoom()

    /**
     * Methode (fonction booléenne) qui verifie si le beamer est charge
     *
     * @return True si une piece est stockee dans le beamer et False sinon
     */
    public boolean BeamerCharged()
    {
        if (this.aChargedRoom != null){
            return true;
        }
        return false;
    } // BeamerCharged()

```

Maintenant, il faut créer dans GameEngine les méthodes charge et fire:

```

private void charge(final Command pInput){
    String vItemName = pInput.getRestOfLine();
    if(!pInput.hasSecondWord()){
        this.aGui.println("Charge what?");
        return;
    }

    Item vItem = this.aPlayer.getItem(vItemName);
    if (vItem == null){
        this.aGui.println("You don't have a " + vItemName + "!");
        return;
    } // cas ou l'objet n'est pas trouve dans l'inventaire

    if (!(vItem instanceof Beamer)){
        this.aGui.println("You can't charge the " + vItemName + "!");
        return;
    } // on utilise instanceof pour verifier que l item est bien un beamer

    Beamer vBeamer= (Beamer) vItem;
    vBeamer.setChargedRoom(this.aPlayer.getCurrentRoom());
    this.aGui.println("The beamer has been charged. If you fire it, you will come back to this room.")
} //charge()

```

```

private void fire(final Command pInput) {
    String vItemName = pInput.getRestOfLine();
    if (!pInput.hasSecondWord()) {
        this.aGui.println("Fire what?");
        return;
    }

    Item vItem = this.aPlayer.getItem(vItemName);
    if (vItem == null) {
        this.aGui.println("You don't have a " + vItemName + "!");
        return;
    }

    if (!(vItem instanceof Beamer)){
        this.aGui.println("You can't fire the " + vItemName + "!");
        return;
    }

    Beamer vBeamer = (Beamer) vItem;
    if (!vBeamer.BeamerCharged()){
        this.aGui.println("The beamer is not charged!");
        return;
    }

    Room vBeamerDestination = vBeamer.getChargedRoom();
    this.aPlayer.setCurrentRoom(vBeamerDestination);
    this.aGui.println("You have been teleported back to the room where you charged the beamer.");

    vBeamer.setChargedRoom(null); // met a zero la charge du beamer
    printLocationInfo();
    if (vBeamerDestination.getImageName() != null) {
        this.aGui.showImage(vBeamerDestination.getImageName());
    }
}

```

pour fire, on ajoute un setter dans player:

```
/**
 * Modifie la piece actuelle du joueur
 *
 * @param pRoom Nouvelle piece
 */
public void setCurrentRoom(final Room pRoom) {
    this.aCurrentRoom= pRoom;
}
```

et finalement, on ajoute les commandes dans intepretCommand:

```
else if ( vCommandWord.equals( "charge" ) )
    this.charge(vCommand);
else if ( vCommandWord.equals( "fire" ) )
    this.fire(vCommand);
```

Ex 7.46

Dans cet exercice on crée une classe TransporterRoom héritant de Room. Si on tente de quitter cette room, on est transporté dans une autre salle choisie aléatoirement.

- on ajoute la room dans createRooms() de GameEngine

```
TransporterRoom vTransporterRoom = new TransporterRoom("in the transporter room", "Images/vOutside.jpg");
```

(l'image n'a pas été changé pour le moment)

- on crée un attribut tableau qui contient les rooms qui pourront être choisies aléatoirement depuis la Transporter Room:

```
private Room[] aExitRooms;
```

- on ajoute un tableau de type Room qui contiendra toute les rooms ou on peut aller a partir de la Transporter Room:

```
Room[] vExitRooms = new Room[]{ vOutside, vFallZone, vReception,
```

- on ajoute l'accès à la TransporterRoom:

```
vRestaurant.setExit("south", vTransporterRoom);
```

- On ajoute la méthode getRandomRoom dans Transporter Room qui va générer une pièce aléatoire après avoir vérifié grâce au if qu'il y a bien une ou des salles définies.

```
/**
 * Retourne une room aleatoire parmi les rooms choisies
 * @return une Room aleatoire
 */
public Room getRandomRoom(){
    if (this.aExitRooms == null || this.aExitRooms.length == 0) {
        return this;
    }
    int vNbSalle = this.vRandom.nextInt(this.aExitRooms.length);
    return this.aExitRooms[vNbSalle];
} //getRandomRoom()
```

Maintenant, on modifie la méthode goRoom dans GameEngine pour prendre en compte le cas où on est dans une TransporterRoom (on ajoute le dernier if).

```
String vDirection = pInput.getSecondWord();
Room vCurrentRoom = this.aPlayer.getCurrentRoom();
Room vNextRoom = this.aPlayer.getCurrentRoom().getExit(vDirection);

if (vNextRoom == null){
    this.aGui.println("There is no door!");
    return;
} // end if

if (vCurrentRoom.getDescription().equals("in the transporter room.")){
    TransporterRoom vTransporterRoom = (TransporterRoom) vCurrentRoom;
    Room vRandomizedRoom = vTransporterRoom.getRandomRoom();
    this.aPlayer.setCurrentRoom(vRandomizedRoom);
}
```

Note: On ajoute quand même une sortie quand on est dans la Transporter Room:


```
vTransporterRoom.setExit("north", vRestaurant);
```

en réalité, même si on tape go north, on ira pas dans le restaurant mais dans une pièce aléatoire.

ex 7.46.1

la commande alea servira lorsque nous sommes dans la transporterRoom et que nous souhaitons aller dans une chambre spécifique. Il est plus simple d'intégrer une HashMap ici, donc dans GameEngine j'ajoute et j'initialise la HashMap aRooms:

```
private HashMap <String, Room> aRooms;  
  
public GameEngine(){ // constructeur par défaut  
    this.aRooms=new HashMap<String, Room>();  
    this.aPreviousRooms= new Stack<Room>();  
    this.createRooms();  
    this.aParser= new Parser();  
}
```

- on met toutes les rooms à l'intérieur:

```
this.aRooms.put("outside", vOutside);  
this.aRooms.put("fallzone", vFallZone);  
this.aRooms.put("reception", vReception);  
this.aRooms.put("waynecabin", vWayneCabin);  
this.aRooms.put("hallway", vHallway);  
this.aRooms.put("majorcabin", vMajorCabin);  
this.aRooms.put("locomotive", vLocomotive);  
this.aRooms.put("langleycabin", vLangleyCabin);  
this.aRooms.put("restaurant", vRestaurant);  
this.aRooms.put("serviceroom", vServiceRoom);  
this.aRooms.put("baggage", vBaggage);  
this.aRooms.put("roof", vRoof);  
this.aRooms.put("transporterroom", vTransporterRoom);
```

- Tout d'abord, on ajoute aléa à la liste de commandes dans CommandWords.
- On ajoute directement dans interpretCommand() de GameEngine le cas où l'on tape alea:


```
else if ( vCommandWord.equals( "alea" ) )
    this.alea(vCommand);
```

- on ajoute la méthode alea:

```
public void alea(final Command pInput)
{
    if (!pInput.hasSecondWord()) {
        this.aGui.println("Alea where?");
        return;
    }

    String vRoomName = pInput.getSecondWord();
    Room vAleaRoom = this.aRooms.get(vRoomName);
    Room vCurrentRoom = this.aPlayer.getCurrentRoom();

    if (vAleaRoom == null) {
        this.aGui.println("The room " + vRoomName + " does not exist!");
        return;
    }

    if (!(vCurrentRoom.getDescription().equals("in the transporter room."))) {
        this.aGui.println("The command 'alea' can only be used if you are in the Transporter Room.");
        return;
    }

    this.aPreviousRooms.push(vCurrentRoom);
    this.aPlayer.setCurrentRoom(vAleaRoom);
    printLocationInfo();
    int vMovesLeft = this.aMaxMoves - this.aPlayer.getMoves();
    this.aGui.println("You have " + vMovesLeft + " moves left.");
    if (vAleaRoom.getImageName() != null) {
        this.aGui.showImage(vAleaRoom.getImageName());
    }
}
```

note: le “second word” sera, dans les cas où on veut aller à Miss Langley’s Cabin par exemple, “langleycabin”.

Exercice supplémentaire: situation gagnante de mon jeu

- Pour gagner le jeu, il faut rassembler 6 items spécifiques que j’ai rassemblé dans un tableau de type String, nommé ItemsToWin dans GameEngine.

```
private final String[] ItemsToWin = {"broken watch", "fountain pen", "handkerchief", "bloody knife", "wayne key", "poisoned wine glass"};
```

- ensuite, on crée la méthode checkVictoryCondition qui va vérifier si les items présents dans l’inventaire sont ceux du tableau ItemsToWin:

si ce n'est pas le cas, on arrête prématurément la méthode car il manque au moins un item. sinon, on affiche le message suivant, qui va attribuer à un attribut booléen aJudgement la valeur true:

```
private boolean aJudgement;
```

```
private void checkVictoryCondition(){
    ItemList inventory = this.aPlayer.getInventory();

    for (String vItemName: ItemsToWin) {
        if (inventory.getItem(vItemName)== null) {
            return;
        }
    }
    this.aGui.println("You have gathered enough evidence.");
    this.aGui.println("What is your judgement, Poirot?");
    this.aGui.println("Type 1 to accuse or 2 to give up:");
    this.aJudgement = true;
}
```

- on vérifie la condition à chaque fois que l'on prend un objet, donc on ajoute a la fin de take() la ligne checkVictoryCondition();
- Finalement, on ajoute dans interpretCommand() le cas où tous les bons items ont été récupérés, la condition est donc satisfaite. Si l'on tape 1, on a gagné, sinon, on a perdu. Si on tape autre chose, on a un message d'erreur.

```
if (this.aJudgement==true){
    if ("1".equals(vInput)) {
        this.aGui.println("You are all the murderers. Did you think me, Detective Poirot, wouldn't be able to realise that?");
        this.aGui.println("You will all be reported to the authorities upon arrival.");
        this.aGui.println("You have won!");
        this.endGame();
    }
    else if ("2".equals(vInput)){
        this.aGui.println("I can't think of anything.");
        this.aGui.println("You lost the game. What a disgrace, Poirot.");
        this.endGame();
    }
    else{
        this.aGui.println("Please type either 1 or 2.");
    }
    return;
}
```

III. Mode d'emploi (si nécessaire, instructions d'installation ou pour démarrer le jeu):

- Avoir installé préalablement le logiciel BlueJ, et lancer le jeu en cliquant sur “new Game()” avec un clic droit sur la classe Game.
- Pour connaître toutes les commandes possibles que l'on peut faire dans le jeu, tapez “test allcommands”.

IV. Déclaration obligatoire anti-plagiat:

- Ce jeu est inspiré du film “Murder on the Orient-Express” sorti en 2017. C'est là d'où m'est venu l'idée de faire une sorte de “Murder Mystery” sous forme de jeu d'aventure. Bien évidemment, je l'ai fortement adapté à ma manière.
- Toutes les images que j'ai utilisées ont été générées par l'IA ChatGPT. J'ai préféré faire cela de telle sorte à ce que je puisse personnaliser mes images le plus possible, les images sur Internet étant trop différentes lorsque l'on change de pièce à pièce, rendant le jeu très peu uniforme.
- L'IA ChatGPT et le site web W3Schools et geeksforgeeks ont également été parfois utilisés pour des explications/précisions sur des notions incomprises, et des suggestions de commandes.
- Je remercie également toute personne m'ayant apporté des éclaircissements sur une certaine notion.